큐

큐의 개요

- 큐(Queue)란?
  - 삽입과 삭제가 다른 장소에서 이루어지는 순서 리스트
  - 삽입 : rear, 삭제 : front
  - FIFO : First-in-First-Out
- 큐의 응용
  - 운영체제(Job scheduling), Spooler
  - 시뮬레이션, 컴퓨터네트워크 등

배열 큐의 구현

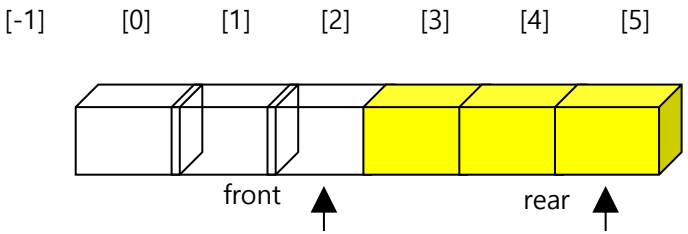
- 큐의 구조체 정의

```
#define MAX_QUEUE_SIZE 100 /* 큐의 최대크기 */
typedef struct {  /* 응용에 따라 정의 */
            int key;
            } element;
element queue[MAX_QUEUE_SIZE];
int rear = -1;
int front = -1;
```

- 큐의 구현 방식 : 선형 큐, 원형 큐

배열 큐의 구현

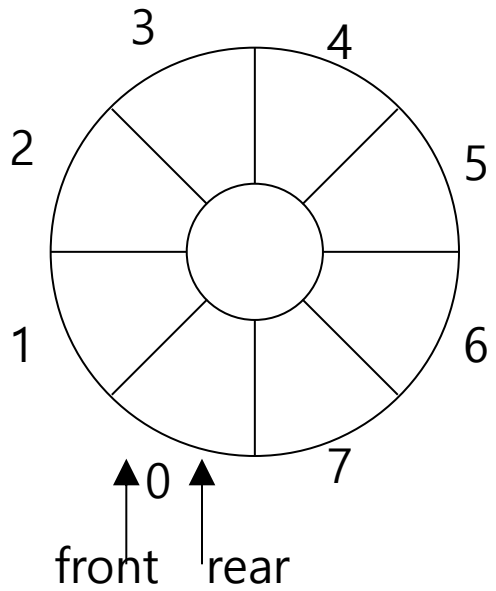
- 선형 큐
  - 단순 선형 배열로 구현
  - 삽입 시 rear 값 증가
  - 삭제 시 front 값 증가
  - Full일 경우 배열 조정



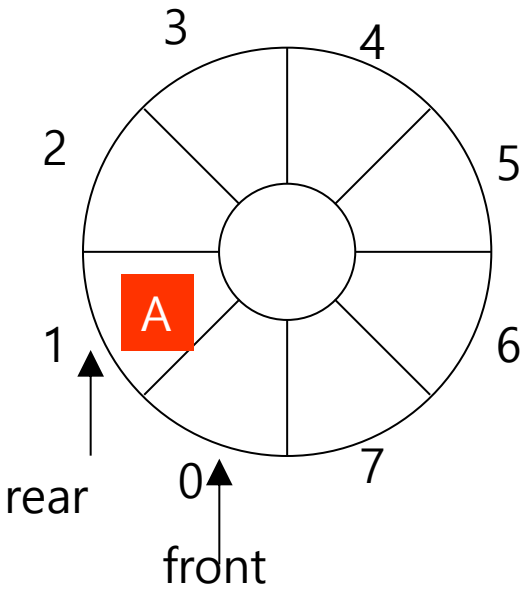
- 해결 방법 : 원형 큐 사용
  - rear = (rear+1) mod MAX_QUEUE_SIZE
  - front = (front+1) mod MAX_QUEUE_SIZE

배열 큐의 구현

- 원형 큐의 예



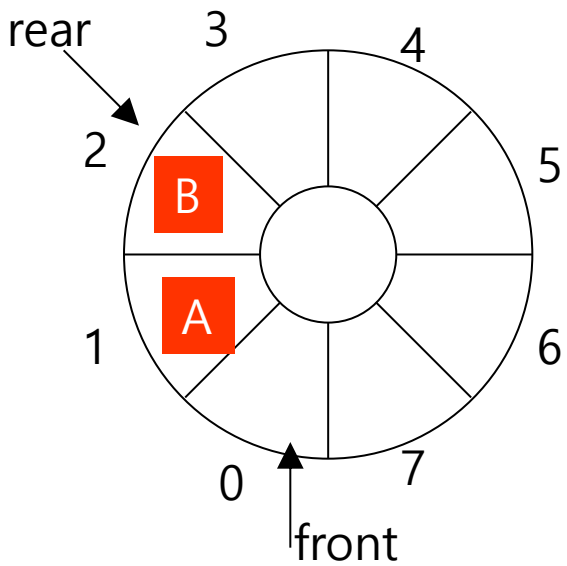
초기상태



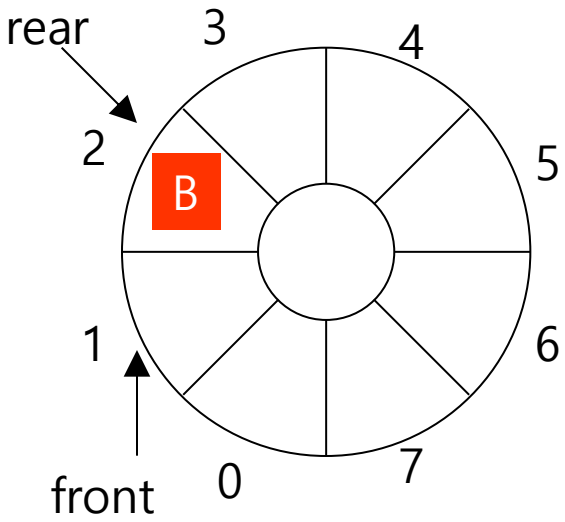
A 삽입

배열 큐의 구현

- 원형 큐의 예

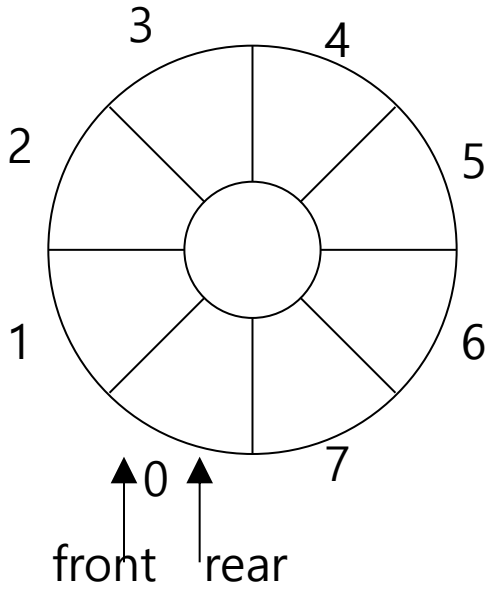


B 삽입



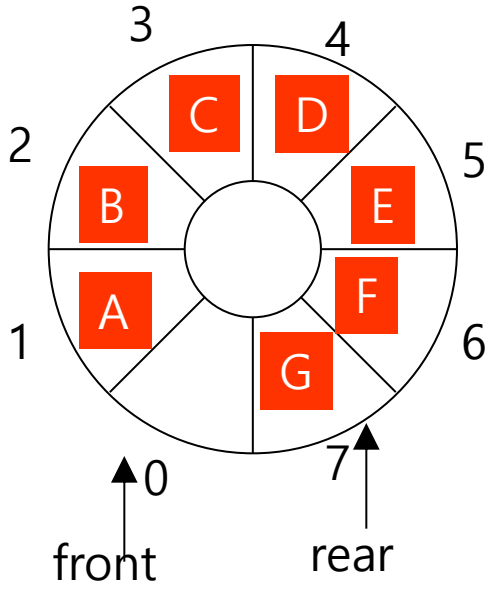
A 삭제

배열 큐의 연산 : 공백상태



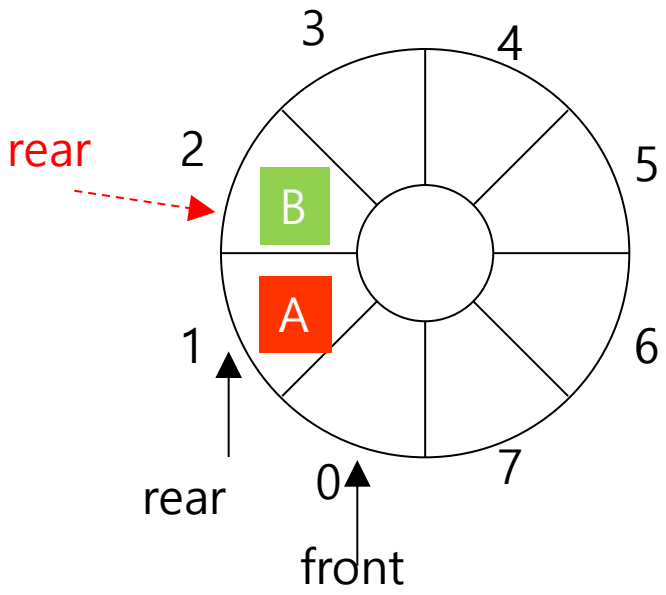

```
int is_empty()
{
    return (front == rear)
}
```

배열 큐의 연산 : 포화상태



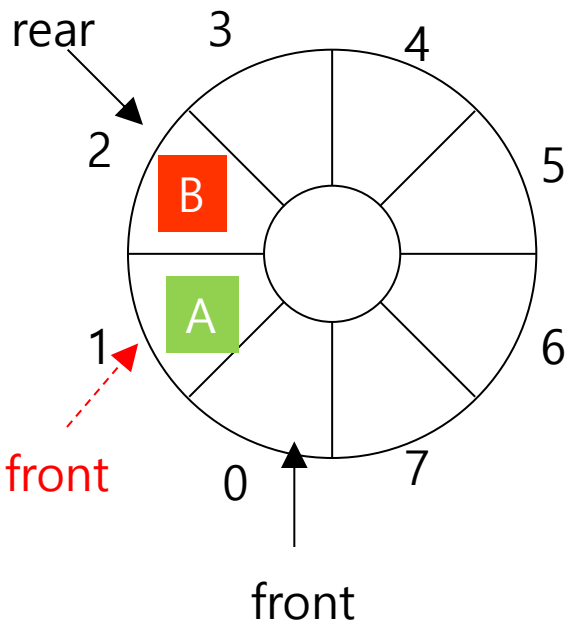

```
int is_full()
{
    return (front == ((rear+1) %
                MAX_QUEUE_SIZE)))
}
```

배열 큐의 연산 : 삽입




```
void enqueue(element item)
{
    if (is_ full())
            error("Queue full);
    rear = (rear+1) %
                MAX_QUEUE_SIZE;
    queue[rear] = item;
}
```

배열 큐의 연산 : 삭제

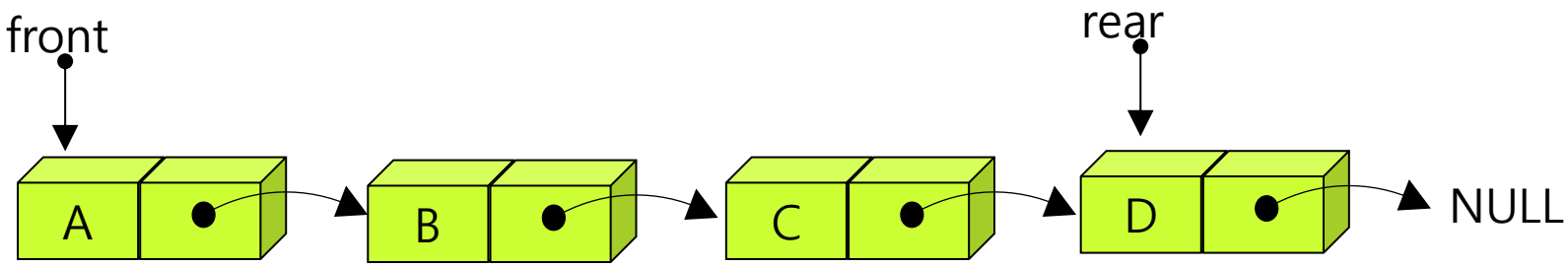



```
element dequeue()
{
    if (is_empty())
            error("Queue empty);
    front = (front+1) %
                MAX_QUEUE_SIZE;
    return queue[front];
}
```

연결리스트 큐의 구현

- 연결리스트 사용
  - 메모리 공간을 동적으로 할당
  - 큐의 포화상태가 의미 없음(메모리 가용)

- 연결리스트를 사용한 큐의 예



  - 초기상태 : front=rear=NULL
  - front : 삭제, rear : 삽입

연결리스트 큐의 구현

- 구조체의 정의

```
typedef struct {  /* 응용에 따라 정의 */
            int key;
            } element;
typedef struct QueueNode {
             element item;
             struct QueueNode *link:
             } QueueNode;
QueueNode *front, *rear;
```

연결리스트 큐의 연산

- 초기화

```
void init()
 {
    front=rear=NULL;
 }
```

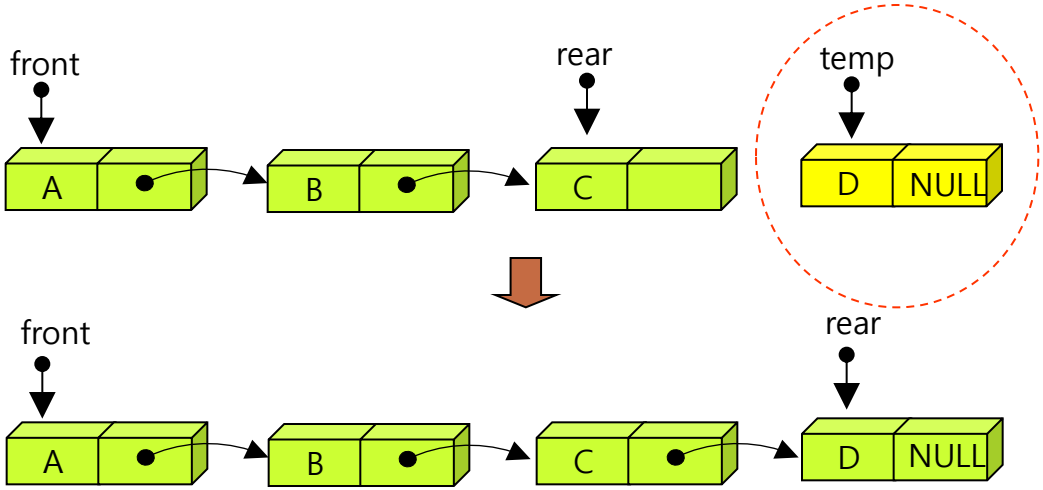
- 포화상태

```
int is_full()
 {
    return 0;
 }
```

- 공백상태

```
int is_empty()
 {
    return (front==null)
 }
```

연결리스트 큐의 연산 : 삽입



case 1 : empty

```
front = temp;
rear = temp;
```

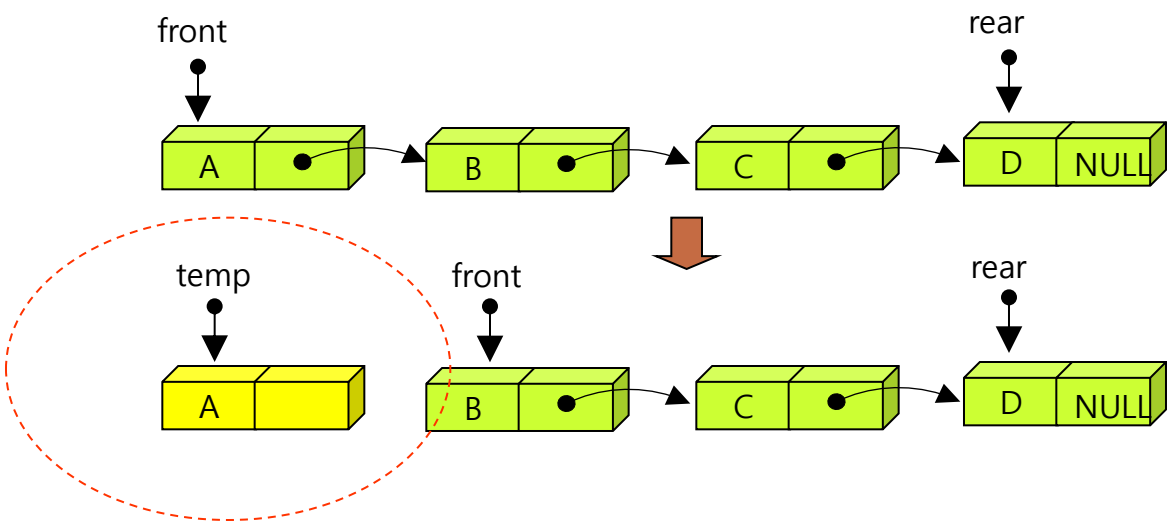
case 2 : not empty

```
rear->link = temp;
rear = temp;
```

연결리스트 큐의 연산 : 삽입

```
void enqueue(element item)
 {
     QueueNode *temp=(QueueNode *) malloc(sizeof(QueueNode));
     temp->item = item;
     temp->link = NULL;
     if (is_empty()) {
                front = temp;
                rear = temp;   }
     else {
                rear->link = temp;
                rear = temp;
     }
 }
```

연결리스트 큐의 연산 : 삭제



case 1 : empty

error("queue empty");

case 2 : not empty

```
temp = front;
item = temp->item;
front = front->link;
```

연결리스트 큐의 연산 : 삭제

```
element dequeue()
 {
    QueueNode *temp = front;
    element item;
    if (is_empty()) error("queue empty");
    item = temp->item;
    front = front->link;
    if (front == NULL) rear = NULL;
    free(temp);
    return item;
 }
```

연결리스트 큐의 연산 : 삭제

```
element dequeue()
 {
    QueueNode *temp = front;
    element item;
    if (is_empty()) error("queue empty");
    item = temp->item;
    front = front->link;
    if (front == NULL) rear = NULL;
    free(temp);
    return item;
 }
```

큐를 사용한 전위식의 계산

- 전위식 : - + × 9 + 2 8 × + 4 8 6 3

큐를 사용한 전위식의 계산

- 알고리즘

▪ 전위식을 스캔하여 토큰 값이 피연산자일 경우 큐에 삽입한다.

▪ 토큰 값이 연산자일 경우 다음 두 경우를 구분하여 처리한다.

- 큐에서 연산자 다음의 두 개의 토큰이 모두 피연산자라면 두 개의 피연산자에 연산자를 취하여 계산한 값을 큐에 삽입한다.

- 그렇지 않을 경우 토큰을 큐에 삽입한다.

큐를 사용한 전위식의 계산

전위식 : - + × 9 + 2 8 × + 4 8 6 3

queue[]	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
값		-	+	×	9	+	2	8	×	+	4	8	6	3					
	f													r					

큐를 사용한 전위식의 계산

전위식 : - + × 9 + 2 8 × + 4 8 6 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18

큐를 사용한 전위식의 계산

```
Algorithm Prefix_eval(prefix_exp)
Input: 전위식이 저장된 배열(prefix_exp)
Output: 전위식을 계산한 결과 값
exp_len <- length(prefix_exp) // 전위식의 길이
for i <- 0 to exp_len-1 do
    Queue_enqueue(prefix_exp[i])
queue_len <- rear - front
while (queue_len != 1) do {
    token <- Queue_dequeue()
    switch (token) {
        case operand : {
            Queue_enqueue(token)
            break }
        case operator : {
            op1 <- queue[front+1]
            op2 <- queue[front+2]
            if (op1 = operand) and (op2 = operand)) {
                Queue_dequeue()
                Queue_dequeue()
                result <- op1 operator op2
                Queue_enqueue(result) }
            else
                Queue_enqueue(token) }
    if (rear > front) queue_len <- (rear-front)
        else  queue_len <- (Queue_size-front-1)+(rear+1)
}
final_value <- Queue_dequeue()
return final_value
```

우선순위 변경

병원 응급실에서는 환자들을 접수 순서대로 진료하지만,
응급 환자가 발생할 경우, 현재 대기 중인 일반 환자보다 항상 먼저 진료를 받아야 한다.
이와 같은 진료 순서를 구현할 수 있는 큐(Queue)의 구조와 동작 방식을 설계하시오.

조건
- 일반 환자는 도착 순서대로 진료한다. (FIFO)
- 응급 환자가 발생하면, 현재 큐의 맨 앞에 삽입되어 즉시 진료된다.
- 단, 큐는 배열 기반 선형 구조를 따른다. (front, rear는 이동 가능)

우선순위 변경

Algorithm Enqueue(queue[], &front, &rear, patient, isUrgent)
Input: 배열 큐 queue[], front와 rear 인덱스, 환자 정보 patient, 긴급 여부 isUrgent
Output: 큐에 환자 삽입

```
if 큐가 가득 찼다면
    출력: "큐 오버플로우" 후 종료
if isUrgent == 1 then
    if front == 0 then
        // 앞에 공간이 없으므로 전체를 오른쪽으로 shift
        for i ← rear downto front do
            queue[i+1] ← queue[i]
        rear ← rear + 1
    else
        front ← front - 1    // 앞쪽 빈 공간 존재 → 그대로 삽입 가능
    queue[front] ← patient   // 긴급 환자 삽입
else
    rear ← rear + 1
    queue[rear] ← patient    // 일반 환자 삽입
```